

Heterogeneous Cognitive Transformers: Computational and Epistemic Coprocessors

Working Draft

August 2026

Abstract

Language-model systems commonly group calculators, formal solvers, databases, and search engines under the single label of tool use. We argue that this grouping hides two different failure modes and therefore two different control problems. A *competence deficit* occurs when the required information is available but a specialized computation would be more reliable or efficient; a *knowledge deficit* occurs when reliable, fresh, or source-specific evidence is absent. We propose a position and an evidence-gated research program in which a Transformer remains the learned semantic and generative substrate while computational and epistemic coprocessors can be engaged by generation-time *semantic interrupts*. An interrupt normalizes an emerging need into typed micro-IR, routes it to a bounded engine, and records the result as provenance-bearing cognitive micro-state before generation continues. This proposal does not claim to invent automatic tool use or active retrieval; rather, it unifies them as two tracks under one runtime contract and asks when automatic, persistent, typed coupling improves over explicit tool calls, planner-executor agents, program-aided reasoning, and upfront or active retrieval. We specify the architecture, locate its novelty relative to prior work, define component-level and end-to-end falsification criteria, and lay out an interleaved sequence of experiments from strict reflexes to a heterogeneous cognitive runtime. Conventional tool calling and upfront retrieval remain supported baselines and fallbacks throughout.

1 Introduction

Transformers are unusually capable semantic interfaces. They map natural language into useful representations, adapt to new tasks through context, and generate coherent continuations. They are nevertheless an awkward place to perform some operations. Exact arithmetic, logical closure, constraint propagation, database aggregation, and source-sensitive factual verification have algorithms and data structures whose reliability does not improve by asking a language model to imitate them for more tokens.

The usual remedy is to give the model tools. This is powerful and should not be discarded. However, the phrase *tool use* covers systems with very different timing, state, and control semantics: a model may explicitly emit an API call, a planner may schedule a solver, a retriever may run once before generation, or a learned token may request evidence during generation. Treating all of these as equivalent obscures a useful systems question:

Can specialized computation and evidence acquisition become ordinary, observable parts of decoding, while the Transformer remains responsible for semantic interpretation, composition, and communication?

We call the resulting design a *heterogeneous cognitive Transformer*. The word *cognitive* describes the role of the external state in the system’s ongoing inference; it is not a claim about consciousness or a biological model. The word *coprocessor* emphasizes a typed, bounded, runtime-managed subsystem that can be activated without requiring the language model to author a complete natural-language tool-use plan.

This paper makes four contributions as a position paper, not as an empirical result:

1. It separates competence deficits from knowledge deficits and defines computational and epistemic coprocessors as twin experimental tracks.
2. It specifies a common interrupt contract: detection, typed normalization, routing, execution, provenance-bearing state update, and controlled reinjection.
3. It distinguishes the proposal from adjacent tool-use, retrieval, and neuro-symbolic systems without claiming that any individual mechanism is new.
4. It turns the architecture into a falsifiable, interleaved research program with matched baselines, component metrics, scaling tests, and an evidence gate before every increase in coupling.

2 Two Deficits, Two Coprocessor Families

2.1 Competence deficits

A competence deficit is present when the premises and operands are already available, but a specialized procedure can perform an operation more reliably, efficiently, or audibly than neural continuation. Examples include arithmetic, symbolic simplification, type checking, graph reachability, logical inference, constraint solving, theorem proving, and bounded simulation. The relevant question is not “what fact is missing?” but “what operation should be performed over the available state?”

A *computational coprocessor* consumes a typed operation and inputs and returns a result, derivation, certificate, counterexample, or explicit failure. Its answer is conditional on its inputs and semantics. Formal derivability does not establish that natural-language premises were translated correctly or that those premises are true.

2.2 Knowledge deficits

A knowledge deficit is present when a pending claim requires information that is missing, stale, source-specific, or insufficiently supported. Examples include current events, private database values, exact quotations, policy versions, and claims for which provenance matters. The relevant question is “what evidence is needed, from which source, and with what freshness?”

An *epistemic coprocessor* consumes a typed information need and returns records, passages, graph neighborhoods, source metadata, timestamps, or an explicit no-evidence result. Retrieval supplies evidence, not truth. Sources can be stale, contradictory, irrelevant, or malicious; the runtime must retain that uncertainty rather than silently converting retrieved text into fact.

The categories are not a forced partition of every request. “Which product had the largest increase in sales last quarter?” may require a database lookup followed by aggregation. The distinction remains useful because it exposes where an error occurred and prevents a calculator result from being treated as source evidence or a retrieved passage from being treated as a proof.

This symmetry is the program’s broadest differentiator. A competence deficit routes toward calculator, logic, SMT, graph, algebra, or code; an epistemic deficit routes toward relational DB,

Table 1: The distinction determines the request, success criterion, and main failure mode. Some tasks compose both families.

	Competence deficit	Knowledge deficit	Composed case
Need	Perform or check an operation	Acquire or verify evidence	Acquire operands, then compute
Typical request	Typed expression, constraints, graph, program	Entity, relation, query, source, freshness	Evidence request followed by typed operation
Primary output	Value, derivation, proof status, counterexample	Record, passage, provenance, evidence status	Provenance-linked derived result
Success test	Semantic and execution correctness	Retrieval relevance, support, freshness, source quality	Both tests plus correct composition
Characteristic risk	Wrong formalization or unsafe execution	Stale, conflicting, missing, or poisoned evidence	Laundering weak evidence through exact computation

lexical index, vector store, knowledge graph, or web search. The shared question generalizes from retrieval timing to which specialized computational or epistemic substrate can assist the evolving neural computation now.

2.3 Neural uncertainty is not epistemic risk

Low next-token confidence is an observable model property; epistemic risk is a property of a pending commitment’s dependence on missing, fresh, source-specific, or verifiable evidence. Neither entails the other. The retrieval track must therefore measure confidence on the actual checkpoint and populate four diagnostic regimes rather than assign them by intuition:

Confidence	Epistemic risk	Diagnostic expectation
High	Low	continue
Low	Low	possible confidence-triggered over-retrieval
High	High	critical semantic-trigger test
Low	High	both confidence and semantic triggers should fire

Recall in the high-confidence, high-risk quadrant tests confident hallucination and stale commitments. False retrieval in the low-confidence, low-risk quadrant tests whether uncertainty is being mistaken for an evidence deficit.

3 Architecture

3.1 The neural plane and the interrupt loop

Let $y_{1:t}$ be the generated prefix and h_t the decoding state available to an interrupt policy. The *neural plane* is the Transformer-based semantic and generative computation. A watcher evaluates whether the current state contains an actionable need:

$$d_t = D(y_{1:t}, h_t, \mathcal{S}_t), \quad d_t \in \{\text{NONE}, \text{COMPUTE}, \text{RETRIEVE}\}.$$

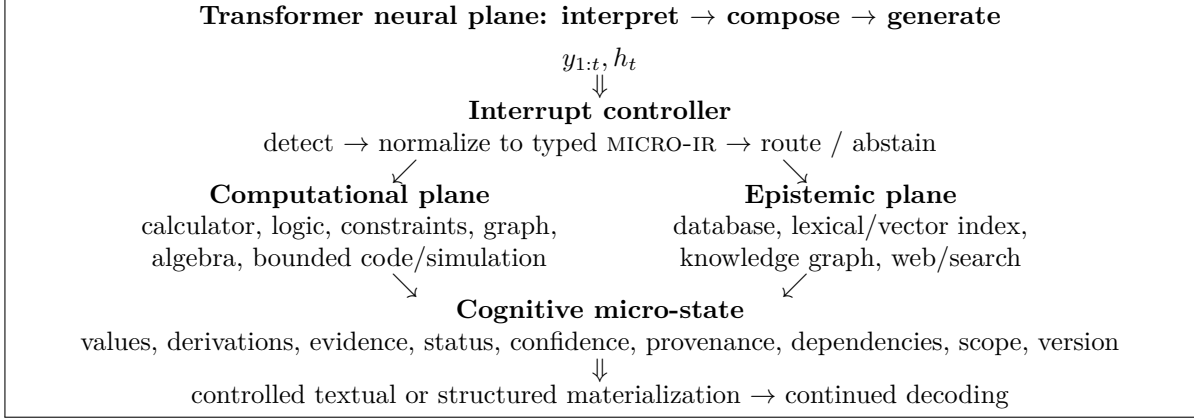


Figure 1: The proposed runtime. Explicit tool calls can enter the same typed execution and state path, and the controller can always abstain.

If $d_t \neq \text{NONE}$, normalization produces a typed event

$$e_t = N(y_{1:t}, h_t, \mathcal{S}_t) = (f, o, a, r, s, c, b),$$

where f is the family, o the operation or information need, a the typed arguments, r the epistemic role, s the scope, c a calibrated confidence, and b a resource budget. A router selects an engine $k = R(e_t)$ or abstains. The engine returns a typed outcome $z_t = E_k(e_t)$, and the state manager performs an auditable update

$$\mathcal{S}_{t+1} = U(\mathcal{S}_t, e_t, k, z_t, \pi_t),$$

where π_t records provenance, versions, timing, and dependencies. A materializer M exposes only the state needed for subsequent generation:

$$y_{t+1} \sim p_\theta(\cdot \mid y_{1:t}, M(\mathcal{S}_{t+1}, h_t)).$$

Detection, normalization, routing, execution, update, materialization, and use are separate failure sites and must be evaluated separately.

3.2 Semantic interrupts are control events

A *semantic interrupt* is an automatically detected opportunity for specialized assistance during generation. It is not necessarily a hardware interrupt, a token emitted by the model, or a pause after every sentence. The implementation may watch completed grammar fragments, parse text, inspect a frozen hidden state, or eventually be trained jointly. Three properties are essential:

1. the event is grounded in the developing inference state rather than only the original query;
2. recognition is separated from typed normalization and execution; and
3. every intervention, abstention, state update, and failure is observable.

Explicit tool invocation remains available. Heavyweight, risky, ambiguous, or user-visible actions should generally require an explicit call or confirmation. Automatic interrupts are most plausible for bounded, reversible, low-latency operations with high-confidence semantics.

3.3 Typed micro-IR

Natural language should not be passed directly to an unrestricted executor when a narrower interface is available. Example computational events include

```
EVAL_ARITH(op=MUL, lhs=37, rhs=48),  
GRAPH_QUERY(op=REACHABLE, src=A, dst=D),  
CHECK_SAT(constraints=C, timeout=100ms).
```

Example epistemic events include

```
RETRIEVE(role=VERIFY_CLAIM, entity=e, relation=r,  
          source=DB, as_of=2026-08-28).
```

The schema should express uncertainty and refusal. A normalizer may return `AMBIGUOUS`; a router may return `NONE`; an executor may return `TIMEOUT`, `UNSUPPORTED`, or `NO-EVIDENCE`. This prevents the runtime from turning weak interpretation into hard state.

3.4 Micro-state and epistemic status

Micro-state is the structured working record produced by interventions. A minimal record contains a stable identifier, type, payload, status, confidence, source or engine version, timestamp, scope, parent dependencies, and the event that created it. Epistemic records should distinguish at least `SUPPORTED`, `CONTRADICTED`, `UNVERIFIED`, `STALE`, `AMBIGUOUS`, and `CONFLICT`. Computational records should retain the formalization and execution status, not just the final value.

State is initially append-only and textually reinjected. Retraction, transactions, branch-local hypotheses, and selective native interfaces are later hypotheses. Introducing them before persistent-state failures are observed would confound the early experiments.

4 Coupling Ladder

The architecture is a ladder of testable couplings rather than a requirement to build the final system at once:

1. **Strict syntax or grammar.** Completed arithmetic or controlled retrieval forms trigger deterministic handlers.
2. **Lexical and syntactic rules.** Transparent patterns recognize a wider set of generation prefixes.
3. **Heuristic routing.** Typed cues select among several engines or evidence sources.
4. **Learned text-level interrupts.** Small detectors and parsers predict whether, what, and where to offload, with abstention and calibration.
5. **Hidden-state routing.** Probes use frozen Transformer states when text alone is insufficient.
6. **Joint interfaces.** The model and runtime co-adapt, potentially using structured or native key/value access.

Each rung must beat the strongest simpler rung under matched capability and cost or solve a measured failure that the simpler rung cannot. “Tighter” is not a synonym for “better”: additional coupling can increase latency, false interventions, opacity, and distribution-shift sensitivity.

5 Illustrative Traces

These examples define intended semantics; they are not implementation results.

Computational reflex. While producing “The total is $37 \times 48 =$ ”, a strict watcher recognizes a completed bounded expression. The normalizer emits `EVAL_ARITH(MUL,37,48)`, the calculator returns 1776, and the state records the expression, result, engine version, and execution status. The materializer exposes “calculator result: 1776” before decoding resumes. A quoted expression, malformed expression, or expression outside the budget must not execute.

Epistemic reflex. While producing “As of August 2026, the current policy limit is”, a watcher marks an `ASSERTION-PENDING` role with a freshness requirement. A source router chooses the versioned policy database; the source returns two conflicting records. The state is `CONFLICT`, not `SUPPORTED`, and the model must cite the disagreement, ask for a governing jurisdiction, or abstain.

Composition. For “Which region grew fastest?”, retrieval first obtains versioned regional values. A computational event then calculates growth rates whose dependencies point to those records. If a source record is retracted, downstream values are marked invalid rather than left as apparently exact facts. This dependency behavior belongs to the transactional-state stage, not the initial reflex experiments.

6 Relationship to Prior Work

The proposal is deliberately adjacent to several mature lines of work. The claim is architectural organization and a falsifiable program across them, not priority over their constituent mechanisms.

6.1 Tool use and agentic control

Toolformer trains a model to decide which API to call, with what arguments, and how to incorporate the result [1]. ReAct interleaves model-written reasoning and actions against external environments [2]. These works establish that tools can improve model behavior and that action can occur within an inference trajectory. Our controller differs mainly in where responsibility is placed: early semantic interrupts are runtime-recognized events with typed state, while model-authored calls remain a fallback and a baseline. This is an empirical comparison, not an assumption that runtime control is superior.

6.2 Program-aided and neuro-symbolic reasoning

PAL delegates generated programs to an interpreter [3]; Logic-LM maps problems to symbolic formulations and invokes deterministic solvers [11]. AlphaGeometry combines a neural language model with a symbolic deduction engine [12], and LeanDojo studies language-model interaction with the Lean theorem prover [13]. Dynamic solver composition routes natural-language problems among multiple logical paradigms [14]. Forethought composes neural and symbolic primitives into inspectable programs [15]. These systems motivate typed, verifiable external computation. The proposed program focuses on incremental generation-time detection, a common state contract spanning computation and retrieval, and controlled progression from reflexes to learned coupling.

6.3 Retrieval-augmented generation

RAG combines parametric generation with non-parametric retrieval [4]. More directly, FLARE actively retrieves during long-form generation by predicting an upcoming sentence, using low-confidence tokens to trigger retrieval, querying with the prediction, and regenerating with evidence [5]. Self-RAG trains the generative LM to retrieve on demand and to emit reflection tokens concerning retrieval relevance, generation support, and response utility [6]. Dynamic and adaptive retrieval methods further study real-time information needs and question-complexity policies [7, 8]; corrective RAG evaluates retrieved evidence and can fall back to web search [9]; multi-source adaptive RAG already studies when, what, and which source to retrieve [10]. Generation-time or learned adaptive retrieval therefore is not novel by itself.

The retrieval track instead asks whether confidence and semantic epistemic risk fail on different cases, whether structurally different evidence substrates require source-specific routing and query semantics, where learned control should live, and whether typed persistent evidence state enables later dependency propagation and retraction.

6.4 Context and runtime management

MemGPT uses operating-system ideas, memory tiers, and interrupts to virtualize context [16]. INFERCEPT addresses scheduling and key/value-cache costs for interrupted augmented inference [17]. These works make clear that interruption and persistent state are also runtime concerns. Progressive Retrieval Attention (PRA), developed as a separate research line in this project, is a later candidate for selectively materializing large typed state; it is neither assumed nor required in Papers 1–3.5.

7 Novelty Claim and Non-Claims

The strongest defensible claim is a conjunction. No single row in Table 4 is asserted to be unprecedented. The research question is whether the conjunction creates measurable value and whether it can be built incrementally without importing all later mechanisms into the first result.

This paper does *not* claim that symbolic systems replace Transformers, that retrieved evidence is true, that formal derivations validate their premises, that hidden-state interfaces are necessarily better than text, or that the complete runtime has been implemented. It also does not claim that the term *semantic interrupt* supersedes prior notions of active retrieval, tool-use tokens, or runtime interruption; it names the common control event used by this program. In particular, neither retrieval during generation nor learned adaptive retrieval is claimed as novel. FLARE and Self-RAG are direct prior art. Nor are relevance/support labels a Paper 4 contribution; Paper 4 is reserved for persistent cross-source provenance, dependencies, transactional scopes, retraction, and backtracking.

8 Falsifiable Program Hypotheses

Let q denote task quality, u unsupported-claim rate, n generated tokens, m model calls, ℓ latency, a external calls, and s active state size. For a system j , define a declared cost-sensitive utility

$$U_j = q_j - \lambda_u u_j - \lambda_n n_j - \lambda_m m_j - \lambda_\ell \ell_j - \lambda_a a_j - \lambda_s s_j,$$

where all λ values are fixed before evaluation and raw metrics are reported alongside U . No single weighting should decide the result; report Pareto frontiers and threshold sweeps.

H1: Reliability. At matched model, task information, and external capabilities, accepted computational interrupts reduce operation errors and accepted epistemic interrupts reduce unsupported factual commitments without an offsetting rise in false interventions. H1 is falsified if component-correct interventions do not improve end-task reliability, or if detection, normalization, routing, or evidence-use errors erase the engine advantage.

H2: Efficiency. For regimes containing repeated bounded operations or emerging information needs, automatic typed assistance yields a better quality–cost frontier than neural emulation, verbose explicit calls, or indiscriminate retrieval. H2 is falsified if gains vanish under matched token, latency, call, and engine-work budgets, or if a simpler baseline weakly dominates the proposed mechanism.

H3: Scaling. As reasoning depth d , problem size p , source count r , factual subgoals g , or state size s increases, a suitable heterogeneous system degrades more slowly than matched homogeneous and explicit-tool baselines. This requires estimating slopes or fitted scaling curves, not comparing one endpoint. H3 is falsified if the proposed system’s error or cost slope is no better, or if routing and state-management overhead grows faster than the saved work.

H4: Small-model leverage. Smaller Transformers receive a larger marginal benefit from correct coprocessor assistance because they can concentrate on recognizing structure and using results rather than emulating every operation. Test the interaction between model size and intervention condition with matched model families and tasks. H4 is falsified if relative gains are flat, favor only larger models, or depend on oracle triggers unavailable to the smaller models.

H5: Typed-state leverage. Persistent typed state improves multi-step use, auditability, and correction once workloads require reuse or revision. H5 is falsified if independent text tool calls match quality and cost, if state is rarely reused, or if stale-state contamination outweighs reuse benefits. Transactional and native interfaces are justified only after this simpler state hypothesis survives.

9 Evaluation Protocol

9.1 Matched baselines

Every empirical paper should include, where applicable: (i) the base model without assistance; (ii) matched prompting; (iii) conventional explicit tool/function calling; (iv) upfront RAG for knowledge tasks; (v) the proposed mechanism; and (vi) oracle trigger, route, normalization, or result-use controls. Strong planner–executor or active-retrieval baselines should be added when the task reaches their intended regime. Baselines must receive the same underlying engine or corpus whenever the scientific question concerns interface or timing. Paper 1.5 must include a FLARE-like confidence condition and confidence/semantic OR and AND ablations. Paper 2.5 must compare universal retrieval, heterogeneous routing, explicit source choice, matched-cost broadcast, and oracle source routing. Paper 3.5 must compare FLARE-like confidence control, Self-RAG-style main-model control, a text sidecar, a matched frozen-LM hidden-state probe, and oracle control. Approximations must be labeled rather than named as canonical systems.

9.2 Component decomposition

An end-to-end score cannot identify whether a coprocessor architecture failed. Each accepted and rejected candidate should be logged through

DETECT → NORMALIZE → ROUTE → EXECUTE/RETRIEVE → UPDATE → MATERIALIZE → USE.

Report trigger precision/recall and false-intervention rate; normalization exact or semantic correctness; route top-1/top- k accuracy and abstention; engine/retrieval correctness; evidence status and provenance correctness; state-update correctness; result-use rate; and final-answer quality. Oracle controls estimate the headroom at each boundary.

9.3 Cost, scaling, and robustness

Report generated and reinjected tokens, model calls, intervention count, source fanout, engine work, wall-clock and engine latency, active and total state size, and monetary cost where meaningful. Sweep problem depth/size, source count, number of factual subgoals, context/state size, and model size. Include hard non-trigger controls, quoted and hypothetical statements, malformed events, ambiguous requests, stale and conflicting evidence, unavailable engines, timeouts, adversarial source text, and incorrect coprocessor outputs.

9.4 Reproducibility and analysis

Pin model, tokenizer, corpus/index, engine, and environment revisions. Preserve raw event traces and machine-readable summaries. Use identity-disjoint splits where templates or entities can leak, multiple seeds for stochastic headline results, confidence intervals, and paired tests where examples are shared. Predeclare primary outcomes and cost weights. Qualitative traces illustrate failure modes but are not evidence of success.

10 Interleaved Research Roadmap

The computational and epistemic tracks alternate so that each begins with the smallest transparent reflex, expands to heterogeneity, and only then learns interrupts. Papers 4–7 integrate mechanisms only when earlier results expose a need.

11 Failure Modes, Safety, and Governance

Automatic intervention creates risks beyond ordinary answer error.

- **False interrupts and missed needs.** Over-triggering wastes work and can alter correct generations; under-triggering preserves the original weakness. Threshold and selective-risk curves are therefore mandatory.
- **Semantic misnormalization.** A correct engine can make an incorrect formalization look authoritative. Store the source span and typed event, expose uncertainty, and use oracle-normalization controls.
- **Unsafe execution.** Prefer bounded domain-specific engines over arbitrary generated code. Enforce time, memory, recursion, expression-size, network, and side-effect budgets.

- **Evidence laundering.** Retrieval relevance is not source reliability, and exact computation over a retrieved value does not strengthen that value’s provenance. Derived records must retain source dependencies.
- **Prompt injection and source poisoning.** Evidence is data, not authority to modify runtime policy. Separate source content from control channels and test adversarial documents.
- **Stale or branch-leaked state.** Hypotheticals, quotations, and abandoned plans must not silently become global facts. Early papers constrain such cases; Paper 4 tests scope and rollback directly.
- **Latency and denial of service.** Interrupt storms and expensive routing can dominate generation. Apply per-event and per-request budgets, deduplication, caching, circuit breakers, and explicit fallback behavior.

Users should be able to inspect when external systems were consulted, what state was created, and which claims depend on it. High-impact external actions remain outside automatic reflexes unless separately authorized; the present program concerns computation and evidence acquisition, not autonomous execution of consequential real-world actions.

12 Limitations and Open Questions

First, the competence/knowledge distinction is operational rather than ontological, and composed tasks can blur it. Its value must be judged by error localization and experimental predictions. Second, observing a generation prefix may be too late to prevent a commitment or too early to formulate the right query. Prospective interruption and retrospective verification should be compared. Third, pauses can perturb decoding independently of the returned result; sham-interrupt controls can estimate this effect. Fourth, a shared event envelope may simplify observability while family-specific semantics still require distinct normalizers, engines, and trust policies.

Fifth, persistent state can become a second context-management problem. Text reinjection is the correct initial baseline, while structured selection, Progressive Retrieval Attention, and native key/value interfaces remain conditional Paper 5 hypotheses. Sixth, learned interrupts may inherit model biases and fail under domain shift; calibration, abstention, and explicit tools must remain available. Finally, the architecture may lose to a well-engineered agent with ordinary calls. That outcome would be scientifically useful: it would identify where tighter coupling adds complexity without value.

13 Conclusion

The proposed research program treats a language model neither as a complete reasoning machine nor as a thin dispatcher. The Transformer remains the central learned substrate for interpreting language, recognizing structure, composing results, and communicating. Specialized systems contribute operations and evidence under explicit typed contracts. Separating competence from knowledge deficits yields twin computational and epistemic tracks; semantic interrupts, micro-IR, and provenance-bearing state provide a common experimental interface.

The proposal succeeds only if this organization produces reproducible gains in reliability, cost, scaling, or small-model leverage against strong simpler baselines. The staged roadmap is therefore part of the architecture: begin with strict reflexes, measure their limits, and add routing, learning, transactions, native access, and co-adaptation only when evidence requires them.

References

- [1] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023. <https://arxiv.org/abs/2302.04761>.
- [2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. <https://arxiv.org/abs/2210.03629>.
- [3] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. *ICML*, 2023. <https://arxiv.org/abs/2211.10435>.
- [4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktaschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS*, 2020. <https://arxiv.org/abs/2005.11401>.
- [5] Z. Jiang, F. F. Xu, L. Gao, Z. Sun, Q. Liu, J. Dwivedi-Yu, Y. Yang, J. Callan, and G. Neubig. Active retrieval augmented generation. *EMNLP*, 2023. <https://arxiv.org/abs/2305.06983>.
- [6] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024. <https://arxiv.org/abs/2310.11511>.
- [7] W. Su, Y. Tang, Q. Ai, Z. Wu, and Y. Liu. DRAGIN: Dynamic retrieval augmented generation based on the information needs of large language models. arXiv:2403.10081, 2024. <https://arxiv.org/abs/2403.10081>.
- [8] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park. Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity. arXiv:2403.14403, 2024. <https://arxiv.org/abs/2403.14403>.
- [9] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling. Corrective retrieval augmented generation. arXiv:2401.15884, 2024. <https://arxiv.org/abs/2401.15884>.
- [10] Q. Zhao, R. Wang, X. Wang, D. Zha, and N. Mu. Towards multi-source retrieval-augmented generation via synergizing reasoning and preference-driven retrieval. arXiv:2411.00689, 2024. <https://arxiv.org/abs/2411.00689>.
- [11] L. Pan, A. Albalak, X. Wang, and W. Y. Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. *Findings of EMNLP*, 2023. <https://arxiv.org/abs/2305.12295>.
- [12] T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nature*, 2024. <https://arxiv.org/abs/2312.11871>.
- [13] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. *NeurIPS*, 2023. <https://arxiv.org/abs/2306.15626>.
- [14] L. Xu, P. Beckmann, M. Valentino, and A. Freitas. Adaptive LLM-symbolic reasoning via dynamic logical solver composition. arXiv:2510.06774, 2025. <https://arxiv.org/abs/2510.06774>.

- [15] V. Bhat, J. Vaghasiya, and E. A. Gonzalez. Forethought: Verifiable reasoning from neurosymbolic primitive programming. arXiv:2607.04096, 2026. <https://arxiv.org/abs/2607.04096>.
- [16] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023. <https://arxiv.org/abs/2310.08560>.
- [17] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang. InferCept: Efficient intercept support for augmented large language model inference. arXiv:2402.01869, 2024. <https://arxiv.org/abs/2402.01869>.

Table 2: Required retrieval comparison. Entries describe the central published mechanisms and the proposed program requirements, not claims that every system is restricted to one deployment configuration.

Dimension	Conventional RAG	FLARE	Self-RAG	Cognitive-coprocessor program
Retrieval timing	Mostly pre-generation	During generation	Adaptive during generation	During generation; upfront retrieval retained
Trigger	Request or harness	Low-confidence predicted tokens	Learned retrieval/reflection behavior	Semantic/epistemic event; later learned
Forward-looking state	Usually no	Predicted upcoming sentence	Adaptive segment-level generation	Prefix/text/semantic state; later latent
Learned trigger	Usually no	No	Yes	Paper 3.5
Multiple retrieval engines	Orchestration-specific	Not central	Not central	Central in Papers 2.5/3.5
Learned source routing	Not central	No	Not central	Paper 3.5
DB/vector/web distinction	Orchestration-specific	No	No	First-class action semantics
Evidence/reflection state	Passages	Retrieved evidence	Relevance/support/utility reflection	Typed persistent epistemic micro-state
Calculator/logic/SMT	Separate tools	No	No	Parallel compute track
Automatic symbolic inference	No	No	No	Paper 2 onward
Transaction/retraction	No	No	Limited critique	Paper 4
PRA/KV micro-state	No	No	No	Conditional Paper 5 interface
Target architecture	Retrieval augmentation	Active retrieval	Self-reflective adaptive RAG	Heterogeneous cognitive runtime

Table 3: Related-work matrix. “Model” means the language model explicitly authors or predicts the action; “runtime” means an external policy can intervene. Entries summarize the primary mechanism, not every possible extension.

Paradigm	Trigger/control	External operation	State returned	Main contrast here
Function/tool calling	Model or orchestrator	Arbitrary APIs	Usually serialized observation	Automatic typed reflex is optional, not required
ReAct / planner-executor	Model-authored plan/action	Tools and environments	Text trajectory	Runtime can recognize bounded needs without a full plan
PAL / program aided	Prompted program generation	Interpreter	Program output	Incremental events and persistent typed state
Neuro-symbolic / theorem proving	Formalization plus solver policy	Logic, proof, geometry	Proof state or solver output	Twin compute/retrieval contract and staged interrupts
Upfront RAG	Input-time query	Usually one retrieval stack	Retrieved context	Needs can emerge after generation begins
FLARE / Self-RAG	Confidence or learned tokens	On-demand retrieval	Passages and reflection signals	Multiple source types plus shared compute/retrieval state
Memory virtualization	Model/runtime memory policy	Store and retrieve context	Memory pages or context	Operation/evidence semantics and provenance, not only capacity
Proposed program	Runtime first; learned later	Heterogeneous compute and retrieval	Typed, scoped, provenance-bearing micro-state	Joint abstraction tested rung by rung

Table 4: Novelty comparison. The proposed entries are program requirements, not completed empirical achievements.

Dimension	Tool/agent systems	Active retrieval	Neuro-symbolic systems	Proposed program
Separate competence from knowledge need	Usually implicit	Knowledge focus	Competence focus	Explicit twin taxonomy
Common typed event contract	API/action schema	Retrieval/reflection schema	Formal problem schema	Shared envelope, family-specific payloads
Generation-time automatic trigger	Sometimes model-authored	Central mechanism	Usually task- or plan-level	Runtime reflex first, learned later
Heterogeneous engine/source routing	Common in agents	Sometimes	Emerging	Both tracks, separately evaluated
Persistent provenance-bearing state	Often text history	Evidence passages/citations	Proof or solver state	Unified micro-state with status and dependencies
Retraction and branch scope	Agent-dependent	Rarely central	Solver-dependent	Explicit later evidence gate
Complexity staged by falsification	Not generally a series constraint	Not generally	Not generally	Required before each coupling rung

Table 5: Per-paper hypothesis, falsification condition, and gate.

Paper	Mechanism	Primary hypothesis	Falsification / negative result	Evidence gate
0	Position and protocol	Twin typed coprocessor tracks form a coherent testable program	No discriminating baselines, metrics, or experiments follow from the distinction	Proceed only with scoped empirical tests
1	Strict reflex calculator	Automatic bounded arithmetic improves reliability or cost over model-only and explicit calls	Explicit calls match the full frontier, or false triggers erase exact execution gains	Heterogeneous compute needs must be observed before Paper 2
1.5	Confidence versus semantic one-source retrieval	Epistemic-risk rules catch evidence needs missed by FLARE-like uncertainty, especially confident commitments	Confidence catches the same cases at equal/lower cost; semantic rules duplicate uncertainty	Multiple sources must address a measured limitation before Paper 2.5
2	Multiple computational engines and typed state	Typed routing/state beats a single engine or repeated explicit calls on mixed tasks	Router overhead/errors dominate; state is not reused	Brittle recognition must limit performance before Paper 3
2.5	Heterogeneous evidence substrates, heuristic routing	Source-specific semantics improve relevance/cost over universal retrieval or broadcast	Oracle routing adds little; universal retrieval or matched-cost broadcast dominates	Oracle heterogeneous routing must win before Paper 3.5
3	Learned computational interrupts	Learned detection/normalization generalizes beyond syntax with calibrated abstention	Heuristics match OOD performance/cost; learned false interrupts dominate	Persistent compute state must create revision pressure before Paper 4
3.5	Main-model versus sidecar epistemic control	Learned control routes heterogeneous sources; transfer/cost reveal where competence should live	Self-RAG-style control dominates, sidecars do not transfer, or source routing adds no value	Both tracks must exhibit concrete provenance/revision failures
4	Transactional unified state	Persistent provenance, dependencies, scope, retraction, and rollback reduce contamination	Append-only state matches quality; rollback errors exceed contamination avoided	Text state must become a measured access bottleneck
5	Structured, PRA, and native interfaces	Selective materialization improves quality-memory-latency tradeoffs at large state	Text serialization remains competitive; native access adds no matched benefit	Co-adaptation requires stable useful interfaces first
6	Joint learning to offload and acquire	Co-adaptation improves formulation/result use and reduces redundant neural work	Gains vanish without coprocessors or under robustness tests; no division-of-labor change is measurable	Integrate only mechanisms with replicated gains
7	Heterogeneous run-time	Validated components compose on mixed workloads with graceful fallback	Integration is dominated by a strong agent/tool baseline or interactions negate component gains	Final synthesis; unsupported components remain excluded